

İleri Dosya İşlemleri

- dosya içinde belirli konuma gitmesi,
- struct verisini kalıcı olarak saklaması,
- kayıt güncellemesi yapması,
- hata kontrolü uygulaması,

İleri Dosya İşlemleri

Fonksiyon	Kısa Açıklama	Tek Satırlık Kullanım (Syntax)
fopen	Dosyayı bir modla açar.	<code>FILE *fp = fopen("dosya.txt", "r");</code>
fclose	Dosyayı güvenli kapatır.	<code>fclose(fp);</code>
fprintf	Dosyaya formatlı yazar.	<code>fprintf(fp, "ID: %d\n", id);</code>
fputs	Dosyaya string yazar.	<code>fputs("Merhaba Dünya", fp);</code>
fscanf	Dosyadan formatlı okur.	<code>fscanf(fp, "%s %d", isim, &yas);</code>
fgets	Dosyadan satır okur.	<code>fgets(dizi, 50, fp);</code>

```
int main() {
    FILE *dosya;
    char okunanSatir[100];
    char isim[20];
    int notu;

    // 1. Yazma (w) modu ile dosya oluşturma ve yazma
    dosya = fopen("notlar.txt", "w");
    if (dosya == NULL) return 1;

    fprintf(dosya, "%s %d\n", "Can", 85); // fprintf kullanımı
    fputs("Basarili bir donem.\n", dosya); // fputs kullanımı
    fclose(dosya); // fclose kullanımı

    // 2. Okuma (r) modu ile dosyadan veri çekme
    dosya = fopen("notlar.txt", "r"); // fopen kullanımı

    // fscanf ile veriyi parçalayarak okuma
    fscanf(dosya, "%s %d", isim, &notu);
    printf("Öğrenci: %s | Notu: %d\n", isim, notu);

    // Bir önceki okumadan kalan satır sonu karakterini temizle
    fgetc(dosya);

    // fgets ile kalan tüm satırı okuma
    fgets(okunanSatir, 100, dosya);
    printf("Yorum: %s", okunanSatir);

    fclose(dosya);
    return 0;
}
```

Öğrenci: Can | Notu: 85
Yorum: Basarili bir donem.

fgets'in çalışma kuralı şudur: "Yol boyunca ne varsa oku ama bir \n (satır sonu) görürsen orada dur."

İleri Dosya İşlemleri

Dosya Açma Modları Kısa Hatırlatma

Temel modlar:

- "r" → okuma
- "w" → yazma
- "a" → ekleme
- "r+" → okuma + yazma
- "w+" → yazma + okuma
- "a+" → ekleme + okuma

Binary modlar:

- "rb", "wb", "ab"
- text modunda metin okunur/yazılır
- binary modunda ham byte düzeyinde veri işlenir

İleri Dosya İşlemleri

Program:

- sayilar.txt adlı bir dosyayı yazma modunda açsın,
- dosyaya 10 20 30 sayılarını yazsın,
- dosyayı kapatsın,
- aynı dosyayı tekrar okuma modunda açsın,
- dosyadaki üç sayıyı okuyup ekrana yazdırsın.

İleri Dosya İşlemleri

```
#include <stdio.h>
```

```
int main(void){  
    FILE *fp = fopen("sayilar.txt", "w");  
    if (fp == NULL) return 1;  
  
    fprintf(fp, "%d %d %d\n", 10, 20, 30);  
    fclose(fp);  
  
    fp = fopen("sayilar.txt", "r");  
    if (fp == NULL) return 1;  
  
    int a, b, c;  
    fscanf(fp, "%d %d %d", &a, &b, &c);  
    printf("%d %d %d\n", a, b, c);  
  
    fclose(fp);  
    return 0;  
}
```

İleri Dosya İşlemleri

Dosya Göstergesi Nedir?

Dosya göstergesi aslında dosyada hangi **bayt (byte)** üzerinde olduğumuzu tutan bir tam sayıdır .

- **Sıfıncı Bayt:** Dosya r veya w moduyla açıldığı an imleç **0. bayttadır**. Bu, dosyanın en başıdır.
- **Otomatik Hareket:** Sen fgetc ile 1 karakter okuduğunda, gösterge otomatik olarak **+1** kayar. 10 karakterlik bir struct okursan **+10** kayar.
- **Yön:** Gösterge genellikle ileri doğru hareket eder, ancak biz müdahale etmezsek asla kendi kendine geri gitmez.

Önemli: Her FILE *fp yapısı, kendi içinde bu "konum" bilgisini saklar.

İleri Dosya İşlemleri

Dosya Göstergesi Nedir?

Dosya bir bütün değildir, bir şerittir. Eğer imlecin nerede olduğunu bilmezseniz şu hatalar kaçınılmaz olur:

- Dosyanın sonuna gelmişsinizdir ama hala okumaya çalışırsınız (EOF hatası).
- Bir veriyi yazdığınızı sanırsınız ama aslında yanlışlıkla başka bir verinin üzerine yazmışsınızdır.
- Dosyayı kapatıp açmadan en başa dönemezsiniz (Eğer rewind bilmiyorsanız).

```
// Dosyada "Merhaba" yazıyor olsun.  
char c1 = fgetc(fp); // 'M' harfini okur. İmleç artık 1. baytta.  
char c2 = fgetc(fp); // 'e' harfini okur. İmleç artık 2. baytta.  
// Program "ben nerede kalmıştım?" diye düşünmez, imleç  
neredeyse oradan devam eder.
```

İleri Dosya İşlemleri

ftell Nedir?

ftell, dosya göstergesinin şu an kaçınıcı byte konumunda olduğunu söyler.:

```
#include <stdio.h>
```

```
int main(void){  
    FILE *fp = fopen("metin.txt", "w+");  
    if (fp == NULL) return 1;  
  
    fprintf(fp, "ABCDE");  
    printf("Konum: %ld\n", ftell(fp));  
  
    fclose(fp);  
    return 0;  
}
```

İmleç **5** olur.

Burada "ABCDE" yazıldıktan sonra
gösterge dosyanın sonuna gider.

ftell(fp) bize mevcut konumu verir.

Eğer `fprintf(fp, "ABCDE\n");` yazmış olsaydın
(sonuna bir `\n` ekleseydin), çıktı **6** olurdu.

İleri Dosya İşlemleri

rewind Nedir?

rewind(fp) dosya göstergesini tekrar dosyanın başına götürür.:

```
#include <stdio.h>
```

```
int main(void){  
    FILE *fp = fopen("metin.txt", "w+");  
    if (fp == NULL) return 1;  
  
    fprintf(fp, "Merhaba");  
    printf("Yazma sonrasi konum: %ld\n",  
ftell(fp));  
  
    rewind(fp);  
    printf("Rewind sonrasi konum: %ld\n",  
ftell(fp));  
  
    fclose(fp);  
    return 0;  
}
```

Bu fonksiyon, özellikle dosyaya yazdıktan sonra tekrar baştan okumak istediğimizde çok kullanılır.

İleri Dosya İşlemleri

fseek Nedir?

fseek, dosya göstergesini istediğimiz yere taşımamızı sağlar.:

Genel kullanım: `fseek(fp, ofset, referans_noktasi);`

```
#include <stdio.h>
```

```
int main(void){  
    FILE *fp = fopen("metin.txt", "w+");  
    if (fp == NULL) return 1;
```

```
    fprintf(fp, "ABCDEFGHJIJ");
```

```
    fseek(fp, 3, SEEK_SET);  
    printf("Konum: %ld\n", ftell(fp));
```

```
    fclose(fp);  
    return 0;
```

```
}
```

Referans noktaları:

SEEK_SET → baştan say

SEEK_CUR → şu an bulunduğun yerden say

SEEK_END → dosya sonundan say

Örneğin: baştan 10 byte ileri

gitmek bulunduğumuz yerden 5 byte

geri gitmek dosyanın sonundan 3

byte önceye gitmek mümkündür.

İleri Dosya İşlemleri

```
#include <stdio.h>
```

```
int main() {
```

```
    FILE *fp = fopen("veriler.txt", "w+");
```

```
    if (fp == NULL) return 1;
```

```
    // 1. Yazma işlemi: ABCDEFGHIJ (10 karakter)
```

```
    fprintf(fp, "ABCDEFGHIJ");
```

```
    printf("Yazma sonrası imlec: %ld\n", ftell(fp)); // Çıktı: 10
```

```
    // 2. Başa dön ve 3. karaktere (D) git
```

```
    // SEEK_SET kullanıyoruz (Baştan 3 bayt ileri)
```

```
    fseek(fp, 3, SEEK_SET);
```

```
    printf("fseek sonrası imlec: %ld\n", ftell(fp)); // Çıktı: 3
```

```
    char c = fgetc(fp); // 3. konumdaki 'D'yi okur
```

```
    printf("Okunan Karakter: %c\n", c); // Çıktı: D
```

```
    // 3. Dosyanın sonundan 2 karakter geriye git (I harfi)
```

```
    fseek(fp, -2, SEEK_END);
```

```
    printf("Sondan 2 geri: %c\n", fgetc(fp)); // Çıktı: I
```

```
    fclose(fp);
```

```
    return 0;
```

```
}
```

İleri Dosya İşlemleri

Program:

- `metin.txt` adlı dosyayı `"w+"` modunda açsın,
- dosyaya `"ABCDEFGHJIJ"` yazsın,
- `fseek(fp, 3, SEEK_SET)` komutunu kullanarak dosya göstergesini baştan 3 byte ileri taşıyın,
- bulunduğu yerden bir karakter okusun ve ekrana yazdırsın.

İleri Dosya İşlemleri

```
#include <stdio.h>
```

```
int main(void){  
    FILE *fp = fopen("metin.txt", "w+");  
    if (fp == NULL) return 1;  
  
    fprintf(fp, "ABCDEFGHJIJ");  
  
    fseek(fp, 3, SEEK_SET); // baştan 3 byte ilerle  
    char ch = fgetc(fp);  
  
    printf("Okunan karakter = %c\n", ch);  
  
    fclose(fp);  
    return 0;  
}
```

İleri Dosya İşlemleri

Rastgele Erişim Nedir?

Temel dosya işleminde genelde dosya baştan sona sırayla okunur.

Rastgele erişimde ise:

- dosyanın istediğimiz bölümüne gideriz,
- sadece ilgili kaydı okuruz,
- gereksiz tüm dosyayı taramayız.

Bu özellikle:

- kayıt sistemleri
- basit veritabanı uygulamaları
- log dosyası analizi

için çok değerlidir.

İleri Dosya İşlemleri

Rastgele Erişim Nedir?

Bellekte (RAM) bir dizi oluşturduğunuzda dizi[784] yazdığınız an bilgisayar oraya anında gider. Bunun sebebi dizideki her elemanın boyutunun aynı olmasıdır.

Eğer dosyadaki kayıtlarımızı da sabit boyutlu (örneğin her biri 100 baytlık struct yapıları) yaparsak, dosyamız artık disk üzerindeki devasa bir diziye dönüşür.

784. kayda gitmek için bilgisayarın yaptığı hesaplama çok basittir:

$$Konum = (Hedef_Kayit_Indeksi) \times Kayit_Boyutu$$

Örneğin, her öğrenci kaydı 50 bayt yer kaplıyorsa ve biz 10. öğrenciyi okumak istiyorsak:

$$(10 - 1) \times 50 = 450. \text{ bayta git.}$$

İşte fseek fonksiyonu bu hesaplamayı bizim yerimize disk üzerinde yapar ve bizi doğrudan o bayta "ışınlar".

İleri Dosya İşlemleri

Rastgele Erişim Nedir?

- **Sıralı Erişim (Text Dosyası):** Bilgisayar 1. kaydı okur, nerede bittiğine bakar (genelde \n karakterini arar), sonra 2.ye geçer... 784. kayda gelene kadar diski defalarca döndürür, binlerce karakteri gereksiz yere okur.
- **Rastgele Erişim (Binary Dosyası):** Bilgisayar hiçbir veriyi okumadan direkt dosya imlecini (pointer) hedef adrese konumlandırır. Bu işlem, dosya ister 100 kayıtlık olsun ister 10 milyon, **neredeyse aynı sürede** gerçekleşir.

Eğer veriler düzensiz bir metin dosyası yerine, sabit boyutlu bloklar halinde (Binary) saklanırsa; devasa verilerin içinden istenilen bilgiye direk erişilebilir.

Bu mantık, bugünkü modern veritabanlarının (SQL, NoSQL vb.) temel çalışma prensibidir.

İleri Dosya İşlemleri

Sabit Boyutlu Kayıt Mantığı

Bir struct yapısının 64 bayt yer kapladığını varsayalım. Dosya içindeki yerleşim tam olarak şöyle görünür:

Kayıt İndeksi	Hesaplama	Dosya Konumu (Bayt)	Durum
0. Kayıt	0 X 64	0	Dosyanın tam başı
1. Kayıt	1 X 64	64	1. kayıt bitti, 2. başladı
2. Kayıt	2 X 64	128	2. kayıt bitti, 3. başladı
n. Kayıt	n X sizeof(Struct)	...	Hedef nokta

İleri Dosya İşlemleri

Struct Verisini Dosyada Saklama

- Bir bilgisayar programı için RAM, geçici bir yazı tahtasıdır; elektrik kesildiği veya program kapandığı an tahta silinir. Dosya (Disk) ise taşta kazınan bir kitabedir. Veriler sadece RAM’de değil, dosyada da saklanabilir.

```
#include <stdio.h>
struct Ogrenci {
    int no;
    char ad[20];
    float ort;
};
int main(void){
    struct Ogrenci o = {101, "Ali", 85.5};

    FILE *fp = fopen("ogrenci.txt", "w");
    if (fp == NULL) return 1;

    fprintf(fp, "%d %s %.2f\n", o.no, o.ad, o.ort);
    fclose(fp);
    return 0;
}
```

İleri Dosya İşlemleri

Struct'ı Text Dosyasından Okuma

- Struct'ı Text Dosyasından Okuma.

```
#include <stdio.h>
struct Ogrenci {
    int no;
    char ad[20];
    float ort;
};
int main(void){
    struct Ogrenci o;
    FILE *fp = fopen("ogrenci.txt", "r");
    if (fp == NULL) return 1;

    fscanf(fp, "%d %19s %f", &o.no, o.ad, &o.ort);

    printf("%d %s %.2f\n", o.no, o.ad, o.ort);
    fclose(fp);
    return 0;
}
```

İleri Dosya İşlemleri

fwrite ve fread Parametreleri

Genel biçim:

```
fwrite(&veri, sizeof(veri_tipi), adet, fp);
```

```
fread(&veri, sizeof(veri_tipi), adet, fp);
```

Örneğin:

- adres
- her elemanın boyutu
- kaç eleman
- hangi dosya

Bu mantığın iyi anlaşılması gerekir.

İleri Dosya İşlemleri

fwrite

- fwrite fonksiyonu, C programlamada veriyi RAM'den alıp olduğu gibi diske "yapıştıran" bir fotokopi makinesi gibidir. Sizin paylaştığınız bu örnek, bir veritabanı kaydı oluşturmanın en saf halidir.
- **Dönüştürme Yok (Sıfır Kayıp):** İprintf kullanıyorsanız, bilgisayar 85.5 sayısını önce "8", "5", ".", "5" karakterlerine çevirmek zorunda kalacaktı. fwrite ise sayıyı bellekteki haliyle (0 ve 1'ler) direkt diske kopyalar. Bu işlem çok daha hızlıdır.
- **Blok Halinde Yazım:** Struct içindeki tüm üyeler (no, ad, ort) tek bir seferde, blok olarak yazılır. Ayrı ayrı fprintf çağırmanıza gerek kalmaz.
- **Sabit Boyut Garantisi:** sizeof(struct Öğrenci) her zaman aynı sonucu verir. Dosyanızdaki her bir öğrenci kaydı dosyada tıpkı bir apartman dairesi gibi **eşit yer** kaplar. Bu, ileride göreceğimiz fseek ile "aradan bir kaydı çekme" işleminin temelidir.

İleri Dosya İşlemleri

fwrite

```
#include <stdio.h>

struct Ogresci {
    int no;
    char ad[20];
    float ort;
};

int main(void){
    struct Ogresci o = {101, "Ali", 85.5};

    FILE *fp = fopen("ogrenci.dat", "wb");
    if (fp == NULL) return 1;

    fwrite(&o, sizeof(struct Ogresci), 1, fp);
    fclose(fp);

    return 0;
}
```

- **&o (Adres):** "Veriyi RAM'in hangi adresinden almaya başlayayım?" (Başlangıç noktası).
- **sizeof(struct Ogresci) (Boyut):** "Her bir veri bloğu kaç bayt?" (Kutunun büyüklüğü).
- **1 (Adet):** "Bu bloktan kaç tane yazayım?" (Sen burada tek bir öğrenci yazdığın için 1 dedin).
- **fp (Hedef):** "Hangi dosyaya yazayım?"

İleri Dosya İşlemleri

fread

- fread, diske kaydedilmiş olan o ham 0 ve 1 yığınınını alır ve belirlediğiniz struct kalıbına (şablonuna) uygun şekilde RAM'e yerleştirir.

```
#include <stdio.h>
```

```
struct Ogresci {
```

```
    int no;
```

```
    char ad[20];
```

```
    float ort;
```

```
};
```

```
int main(void){
```

```
    struct Ogresci o;
```

&veri void *ptr Nereye? (Okunan veri RAM'de hangi adrese yazılsın?)

sizeof(...) **size_t** size Ne kadar büyük? (Her bir bloğun bayt boyutu nedir?)

Adet **size_t** nmembrKaç tane? (Arka arkaya kaç blok okunacak?)

fp FILE *streamNereden? (Hangi dosyadan veri çekilecek?)

```
FILE *fp = fopen("ogrenci.dat", "rb");
```

```
if (fp == NULL) return 1;
```

```
fread(&o, sizeof(struct Ogresci), 1, fp);
```

```
printf("%d %s %.2f\n", o.no, o.ad, o.ort);
```

```
fclose(fp);
```

```
return 0;
```

```
}
```

İleri Dosya İşlemleri

Kayıt Güncelleme: Neden Binary Dosya?

Bir dosyada binlerce kayıt olduğunu ve sadece 500. sıradaki öğrencinin notunu güncellemek istediğimizi düşünelim.

1. Text Dosyasındaki "Kâbus" (Sıralı Erişim)

Text dosyalarında satırlar farklı uzunlukta olabilir (Biri "Ali 10", diğeri "Abdülrezzak 100").

- **Sorun:** Bir veriyi değiştirdiğinizde (örneğin "10" yerine "100" yazdığınızda), o satır uzadığı için dosyanın geri kalan her şeyi sağa kayar.
- **Çözüm:** Dosyanın tamamını RAM'e oku, değişikliği yap ve **tüm dosyayı baştan sona tekrar yaz**. Bu, 1 milyon kayıta korkunç bir zaman kaybıdır.

İleri Dosya İşlemleri

Kayıt Güncelleme: Neden Binary Dosya?

2. Binary Dosyasındaki "Büyü" (Rastgele Erişim)

Binary dosyalarda her struct dosyada **tıpkı bir apartman dairesi gibi** sabit yer kaplar.

- **Çözüm:** Hiçbir şeyi kaydırmanıza gerek yok. Sadece hedef bayta "ışınlanın" (fseek) ve yeni veriyi eski verinin tam üstüne "yapıştırın" (fwrite).

Kayıt Güncelleme Örneği

```
#include <stdio.h>
struct Ogresci {
    int no;
    char ad[20];
    float ort;
};
int main(void){
    FILE *fp = fopen("ogr.dat", "r+b");
    if (fp == NULL) return 1;

    struct Ogresci o;

    fseek(fp, 1 * sizeof(struct Ogresci), SEEK_SET); // 2. kayda git
    fread(&o, sizeof(struct Ogresci), 1, fp);

    o.ort = 95.0;

    fseek(fp, -1 * (long)sizeof(struct Ogresci), SEEK_CUR);
    fwrite(&o, sizeof(struct Ogresci), 1, fp);
    fclose(fp);
    return 0;
}
```

fseek ile Dosyanın en başına (**SEEK_SET**) git,
oradan **1 tane** öğrenci boyutu kadar ileri zıpla.

fread yaptığımızda imleç ileri gitmişti. Eğer şimdi direkt fwrite yaparsak, değişikliği **yanlışlıkla bir sonraki öğrencinin** üzerine yazarız.

- **SEEK_CUR**: "Şu an olduğun yerden" demek.
- **-1 * ...**: "Bir öğrenci boyutu kadar **geriye** git" demek. Bu komutla imleci tekrar az önce okuduğumuz öğrencinin başladığı yere geri çektik.

İleri Dosya İşlemleri

Aşağıdaki alanlara sahip bir struct Öğrenci tanımlayınız:

- no
- ad
- ort

C dilinde bir program yazınız.

Program:

- bir adet struct Öğrenci değişkenine örnek değerler atayınız,
- bu struct verisini ogr.dat adlı dosyaya binary olarak yazınız,
- dosyayı kapatıp tekrar açınız,
- veriyi başka bir struct değişkenine geri okuyunuz,
- okunan bilgileri ekrana yazdırınız.

```
#include <stdio.h>
struct Ogresci {
    int no;
    char ad[20];
    float ort;
};
int main(void){
    struct Ogresci o1 = {101, "Ali", 85.5};
    struct Ogresci o2;

    FILE *fp = fopen("ogr.dat", "wb");
    if (fp == NULL) return 1;

    fwrite(&o1, sizeof(struct Ogresci), 1, fp);
    fclose(fp);

    fp = fopen("ogr.dat", "rb");
    if (fp == NULL) return 1;

    fread(&o2, sizeof(struct Ogresci), 1, fp);
    printf("%d %s %.1f\n", o2.no, o2.ad, o2.ort);
    fclose(fp);
    return 0;
}
```

İleri Dosya İşlemleri

Neden Hata Yönetimi Gerekli?

Dosya işlemleri hata üretmeye çok açıktır:

- dosya bulunamayabilir
- dosya açılamayabilir
- okuma başarısız olabilir
- dosya beklenenden kısa olabilir
- format bozuk olabilir

Bu yüzden sadece “çalışan kod” değil, **sağlam kod** yazmak önemlidir.

İleri Dosya İşlemleri

NULL Kontrolü

fopen başarısız olursa NULL döner.

Kısa kod

```
FILE *fp = fopen("veri.txt", "r");  
  
if (fp == NULL) {  
    printf("Dosya acilamadi!\n");  
    return 1;  
}
```

Bu kontrol yapılmazsa program çökmeye veya beklenmedik davranış göstermeye açık olur.

İleri Dosya İşlemleri

perror Kullanımı

perror, sistemin ürettiği hata mesajını yazdırır.

Kısa kod

```
#include <stdio.h>
```

```
int main(void){
```

```
FILE *fp = fopen("olmayan.txt", "r");
```

```
if (fp == NULL) {
```

```
perror("Dosya hatasi");
```

```
return 1;
```

```
}
```

```
return 0;
```

```
}
```

İleri Dosya İşlemleri

perror Kullanımı

perror, sistemin ürettiği hata mesajını yazdırır.

Kısa kod

```
#include <stdio.h>
```

```
int main(void){
```

```
FILE *fp = fopen("olmayan.txt", "r");
```

```
if (fp == NULL) {
```

```
perror("Dosya hatasi");
```

```
return 1;
```

```
}
```

```
return 0;
```

```
}
```

```
#include <stdio.h>

int main(void) {
    FILE *kaynak = fopen("asli.txt", "r");
    FILE *kopya = fopen("yedek.txt", "w");

    char satir[256]; // Bizim taşıma kutumuz (Buffer)

    // Dosyalar düzgün açıldı mı kontrolü
    if (kaynak == NULL || kopya == NULL) {
        printf("Hata: Dosyalar acilamadi!\n");
        return 1;
    }
    // TAŞIMA BANDI (Kopyalama Döngüsü)
    // fgets NULL dönene kadar (yani dosya bitene kadar) oku
    while (fgets(satir, sizeof(satir), kaynak) != NULL) {
        // Okunan satırı hiç değiştirmeden diğer dosyaya yapıştır
        fputs(satir, kopya);
    }
    // 3. ADIM: Temizlik
    fclose(kaynak);
    fclose(kopya);
    printf("Kopyalama basariyla tamamlandi. 'yedek.txt' olusturuldu.\n");
    return 0;
}
```

Filtreleme

```
#include <stdio.h>
#include <string.h>
```

```
int main(void) {
    FILE *kaynak = fopen("log.txt", "r");
    FILE *hedef = fopen("hatalar.txt", "w");
    char satir[256];
```

bellekte (RAM) 256 karakterlik boş bir yer ayır, adını da 'satir' koy.

```
    if (kaynak == NULL || hedef == NULL) return 1;
```

```
    // TAŞIMA BANDI BAŞLIYOR
```

```
    while (fgets(satir, sizeof(satir), kaynak) != NULL) {
        // SÜZGEÇ: Eğer satırda "HATA" kelimesi varsa...
```

```
        if (strstr(satir, "HATA") != NULL) {
```

```
            fputs(satir, hedef); // Sadece bu satırı hedef dosyaya yaz
```

```
        }
```

```
    }
```

```
    fclose(kaynak);
```

```
    fclose(hedef);
```

```
    printf("Filtreleme tamamlandı. Hatalar 'hatalar.txt' dosyasına yazıldı.\n");
```

```
    return 0;
```

```
}
```

strstr ismi, "**string inside string**" (string içinde string) ifadesinden gelir. <string.h> kütüphanesi ile kullanılır.

- **1. Parametre (satir):** Aramanın yapılacağı ana metin.
- **2. Parametre ("HATA"):** Aranılan kelime.

```
#include <stdio.h>
```

```
#include <ctype.h> // toupper() fonksiyonu için
```

Dönüştürme

```
int main(void) {
```

```
    FILE *girdi = fopen("kucuk.txt", "r");
```

```
    FILE *cikti = fopen("BUYUK.txt", "w");
```

```
    char satir[256];
```

bellekte (RAM) 256 karakterlik boş bir yer ayır, adını da 'satir' koy.

```
    if (girdi == NULL || cikti == NULL) return 1;
```

fgets fonksiyonu her seferinde dosyadan bir satır alır ve senin satir isimli sepetine koyar..

```
    while (fgets(satir, sizeof(satir), girdi) != NULL) {
```

```
        // İŞLEME HATTI: Satırdaki her karakteri tek tek kontrol et
```

```
        for (int i = 0; satir[i] != '\0'; i++) {
```

```
            satir[i] = toupper(satir[i]); // Karakteri büyüt
```

```
        }
```

```
        fputs(satir, cikti); // Dönüşmüş satırı yaz
```

```
    }
```

```
    fclose(girdi);
```

```
    fclose(cikti);
```

```
    printf("Donusturme bitti. Her sey artik buyuk harf!\n");
```

```
    return 0;
```

```
}
```

while döngüsü içeriye bir satır getirdiğinde, for döngüsü hemen işe koyulur. Bu döngü, getirilen o tek satırın içindeki her bir harfle tek tek ilgilenir. int i = 0; "Satırın ilk harfinden (0. indeksten) başla.

```
#include <stdio.h>
```

```
int main(void) {
```

```
    FILE *f1 = fopen("bolum1.txt", "r");
```

```
    FILE *f2 = fopen("bolum2.txt", "r");
```

```
    FILE *hedef = fopen("kitap_tamami.txt", "w"); // Baştan yaz
```

```
    char satir[256];
```

```
    if (f1 == NULL || f2 == NULL || hedef == NULL) return 1;
```

```
    // 1. Dosyayı aktar
```

```
    while (fgets(satir, sizeof(satir), f1) != NULL) {
```

```
        fputs(satir, hedef);
```

```
    }
```

```
    // 2. Dosyayı aktar (Dikkat: hedef dosya 'w' modunda açık kalmaya devam ediyor)
```

```
    while (fgets(satir, sizeof(satir), f2) != NULL) {
```

```
        fputs(satir, hedef);
```

```
    }
```

```
    fclose(f1);
```

```
    fclose(f2);
```

```
    fclose(hedef);
```

```
    printf("Birlestirme basarili. Iki dosya artik tek vucut.\n");
```

```
    return 0;
```

```
}
```